

1. Approach for Building the Feature

The recommended approach is to use a **community Capacitor Health Plugin** that abstracts the complexity of the underlying native platforms.

Platform	Native API	Unified Capacitor Target
iOS	HealthKit	Capacitor Health Plugin
Android	Health Connect (Modern) & Google Fit (Legacy)	Capacitor Health Plugin
Samsung	Integrated into Health Connect/Google Fit ecosystem.	Capacitor Health Plugin

Recommended Tool: Capacitor Health Plugin

Instead of writing custom native code for Apple HealthKit (iOS) and Google Fit/Health Connect (Android), you will install a third-party Capacitor plugin that exposes a single TypeScript API, such as **@capgo/capacitor-health** or **capacitor-health** (search the Capacitor plugin registry for the most maintained option).

This plugin will handle the following for you:

1. **Bridging:** Exposing a simple function like `requestPermissions()` in your TypeScript code.
2. **Native Implementation:** On iOS, it calls **HealthKit**. On Android, it targets the newer **Health Connect** API (Google is deprecating the original Google Fit APIs).
3. **Data Normalization:** Converting platform-specific data types (e.g., Apple's `HKQuantityTypeIdentifierStepCount` and Android's `StepsRecord`) into consistent TypeScript objects for your application logic.

Your overall flow in the TypeScript code will be:

1. Check if the Health API is available (`isHealthAvailable()`).
2. **Request Permissions** (`requestPermissions()`).
3. **Query Data** (e.g., `queryAggregated()` for total steps over a date range).
4. Process the data in your app and send it to your backend.

2. Health Data Permission Flow

The Operating Systems have a strict, highly regulated flow to ensure user privacy and consent for sensitive health data. You must follow these steps on both platforms:

A. iOS (HealthKit) Permission Flow

iOS uses **HealthKit**, a centralized data store accessible only by native apps with user permission.

Step	Action	Description
1. Entitlements	Project Setup	You must enable the HealthKit Capability in your Xcode project settings.
2. Usage Descriptions	Native Code Modification	You must add two privacy keys to your Info.plist file, explaining <i>why</i> you need to read and write data. This text is shown to the user.
	NSHealthShareUsageDescription	Required for reading data (sharing).
	NSHealthUpdateUsageDescription	Required for writing data (updating).
3. Request	Runtime Code Call	When the user initiates the feature (e.g., clicks a "Connect to Apple Health" button), your app calls the plugin's requestPermissions() method, specifying which data types (steps, heart rate, etc.) you need to read and write .
4. User Prompt	OS Modal	Apple's system modal appears. The user can grant or deny access

		to each specific data type individually.
5. Data Access	Result Handling	After the prompt, HealthKit only makes available the data types the user explicitly granted. Crucially, HealthKit will not tell you if a user denied permission; it only tells you if the attempt to request authorization succeeded.

B. Android (Health Connect) Permission Flow

Android's modern standard is **Health Connect**, which acts as a data hub between various health apps (including Google Fit and Samsung Health).

Step	Action	Description
1. Dependencies	Project Setup	You must configure your native Android project to use the Health Connect SDK (this is usually handled by the Capacitor plugin).
2. Manifest Permissions	Native Code Modification	You must manually declare the required health permissions (<code>READ_STEPS</code> , <code>WRITE_HEART_RATE</code> , etc.) in your AndroidManifest.xml file.
3. Rationale Activity	Native Code/Play Console	Google requires you to have an activity that shows the rationale (your privacy policy) for the permissions, which is often shown when users access permissions via the Health Connect app. You must also declare permissions in the Google Play Console .

4. Request	Runtime Code Call	Your app calls the plugin's <code>requestPermissions()</code> method, again specifying the exact read/write permissions it needs.
5. User Prompt	Health Connect Modal	The user is taken to a Health Connect system screen where they can see which apps are requesting access and grant/deny permissions.
6. Data Access	Result Handling	You must periodically check if the user still has permissions granted, as they can revoke them at any time from the Health Connect app.

This process ensures your app complies with platform-specific rules before accessing sensitive user data.

The video below explains how to set up the iOS side of this integration for your Capacitor app.